

Comunicación por variables compartidas y sincronización

- **Introducción**
 - Sincronización con condiciones
 - Exclusión mutua
- **Sincronización**
 - Espera activa
 - Semáforos
 - Problemas de bloqueo
- **Semáforos en POSIX 1003.1b**
 - Creación, acceso y destrucción de semáforos
 - Sincronización con semáforos POSIX 1003.1b
- **Memoria compartida en POSIX 1003.1b**
- **Sincronización para múltiples hilos en POSIX 1003.1c**
 - Mutex
 - Variables de condición
- **Variables compartidas y conceptos de alto nivel**
 - Regiones críticas condicionales
 - Monitores
 - Objetos protegidos de ADA

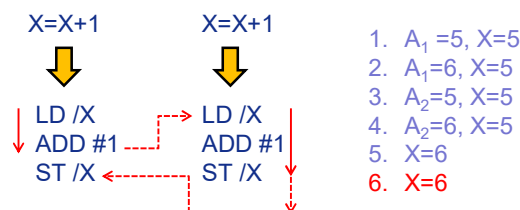
11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

1

Introducción

- **Variables compartidas y sincronización**
 - La comunicación por variables compartidas **no incluye sincronización**
- **Sincronización**
 - Sincronización con condiciones: Esperar determinado acontecimiento que sucede en otra tarea
 - Sincronización por exclusión mutua: Esperar a que los datos compartidos estén disponibles (exclusión mutua)
- **Exclusión mutua:**
 - Necesaria para que las operaciones sobre los datos compartidos sean **atómicas** (indivisibles)
 - Asegura la coherencia de los datos compartidos
 - Código en exclusión mutua: **Sección crítica**
 - Ejemplo:



1. $A_1 = 5, X = 5$
2. $A_1 = 6, X = 5$
3. $A_2 = 5, X = 5$
4. $A_2 = 6, X = 5$
5. $X = 6$
6. $X = 6$

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

2

Espera activa (“busy wait”)

- **Inconveniente básico:** El proceso que espera realiza continuamente una actividad que no es útil.
- **Sincronización con condiciones:**

T1: (consumidor) Hacer mientras flag = 0 Fin hacer <realizar operaciones>	T2: (productor) <realizar operaciones> flag = 1
---	--

- **Exclusión mútua:** No pueden evitarse fácilmente los fallos
 - **Primer caso:** si los dos detectan el flag contrario a 1, **nunca accederán a los datos** (“livelock”):

T1: flag1 = 1 /* La tarea 1 quiere entrar */ Mientras flag2 = 1 /* Primero espera a T2 */ Fin hacer <sección crítica> flag1 = 0 /* T1 ha acabado */ <resto de operaciones>	T2: flag2 = 1 /* La tarea 2 quiere entrar */ Mientras flag1 = 1 Fin hacer <sección crítica> flag2 = 0 /* T2 ha acabado */ <resto de operaciones>
--	--

- **Segundo caso:** si los dos detectan el flag contrario a 0, **no hay exclusión mútua:**

T1: Mientras flag2 = 1 /* Primero espera a T2 */ Fin hacer flag1 = 1 /* La tarea 1 toma posesión */ <sección crítica> flag1 = 0 /* T1 ha acabado */ <resto de operaciones>	T2: Mientras flag1 = 1 /* Primero espera a T1 */ Fin Hacer flag2 = 1 /* La tarea 2 toma posesión */ <sección crítica> flag2 = 0 /* T2 ha acabado */ <resto de operaciones>
---	---

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

3

Semáforos

- Concepto **teórico**, con diversas implementaciones
- Elementos que componen un semáforo:
 - **Valor entero no negativo**, no accesible directamente
 - Operación de **espera, wait**: Si el valor es cero, la tarea es bloqueada hasta que otra tarea lo incremente con **signal**. Cuando el valor del semáforo es mayor que cero, es decrementado y la tarea continúa (si había llegado a bloquearse):

Si **cuenta** == 0, esperar cuenta > 0
 Decrementar **cuenta**

- Operación de **señalización, signal**: Incrementa el valor del semáforo en una unidad (y por tanto, puede desbloquear tareas que están esperando en **wait**).

Incrementar **cuenta**

- **Importante:** Las operaciones se realizan de manera **atómica**, indivisible.

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

4

Semáforos

- Sincronización con una condición (**sem_cond** inicializado a 0):

T1: /* Espera la condición
generada por T2 */
<.....>
wait(sem_cond)
<realizar operaciones>
<.....>

T2: /* Genera la condición
que espera T1 */
<.....>
<realizar operaciones>
signal(sem_cond)
<.....>

- Exclusión mútua (**sem_excl** inicializado a 1):

T1:
<.....>
wait(sem_excl) /* T1 espera a que T2 salga
de su sección crítica */
<.sección crítica>
signal(sem_excl) /* T1 permite acceso a T2 */
<.....>

T2:
<.....>
wait(sem_excl) /* T2 espera a que T1 salga
de su sección crítica */
<.sección crítica>
signal(sem_excl) /* T2 permite acceso a T1 */
<.....>

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

5

Semáforos: Problemas de bloqueo

- Bloqueo mútuo (“**deadlock**”):

T1:	T2:
wait (sem1)	wait (sem2)
<.....>	<...>
wait (sem2)	wait (sem1)
<.....>	<.....>
signal (sem2)	signal (sem1)
<.....>	<...>
signal (sem1)	signal (sem2)

- T1 es expulsada antes de ejecutar el segundo **wait**, y después de ejecutar el primero
- T2 pondría **sem2** a 0, y quedaría bloqueada en su segundo **wait**
- Cuando T1 continúe, no podrá pasar del segundo **wait** tampoco
- La condición de bloqueo permanece indefinidamente

- Inanición (“**starvation**”):

- El algoritmo utilizado por el sistema operativo para escoger a el proceso que es desbloqueado lleva a algunos a la suspensión por **tiempo ilimitado** o con **límites indefinidos**

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

6

Semáforos POSIX (1003.1b)

- Semáforos “con nombre” (“named”):
 - Semáforos con cuenta
 - Objetos de sistema operativo, accesibles a través de su nombre por cualquier proceso con permiso adecuado

```

#include <semaphore.h>

/* Creación, acceso y destrucción */
sem_t *sem;
sem = sem_open("/sem27",
               O_CREAT | O_EXCL, S_IRWXU, 1);
<....>
sem_close(sem);
sem_unlink("/sem27");
<....>

```
- Semáforos en memoria:
 - Semáforos con cuenta
 - Objetos creados en memoria compartida, sin nombre
 - Accesibles por procesos que comparten tal zona de memoria

```

/* Creación, acceso y destrucción */
#include <semaphore.h>

sem_t *psem;

memoria = <inicio de memoria compartida>;
psem = (sem_t *)memoria;
sem_init(psem, 1, 1);
<....>
sem_destroy(psem);

```

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

7

Semáforos POSIX (1003.1b)

- Operaciones básicas:
 - Equivalente de “wait”: **sem_wait**
 - Equivalente de “signal”: **sem_post**
 - El funcionamiento es **el mismo** para ambos tipos:

```

#include <semaphore.h>

sem_t *semafo;

semafo = sem_open(....);
<....>
sem_wait(semafo);
<sección crítica>
sem_post(semafo);
<...>
sem_close(semafo);

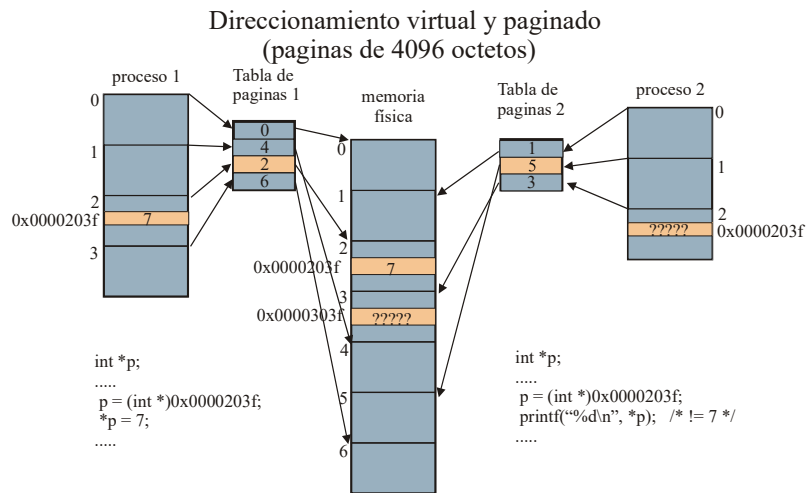
```
- Otras operaciones:
 - Espera no bloqueante: **sem_trywait**
 - Obtención de la cuenta actual: **sem_getvalue**

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

8

Direccionamiento virtual y paginado

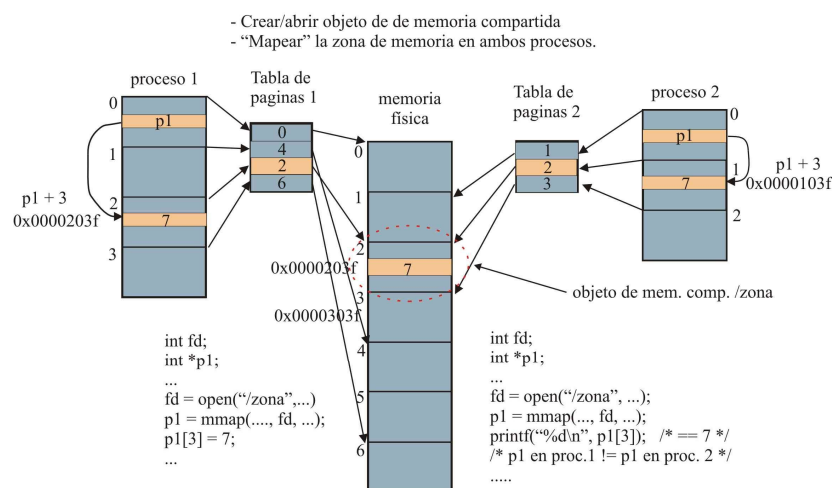


11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

9

Memoria compartida en POSIX 1003.1b



11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

10

Memoria compartida en POSIX 1003.1b

- Características básicas:
 - La zona de memoria debe aparecer (ser “mapeada”) en el espacio de direccionamiento virtual de los procesos que la comparten
 - Mecanismo común con los archivos (`_POSIX_MAPPED_FILES` y `_POSIX_SHARED_MEMORY_OBJECTS`)
 - Conviene manejar siempre un número entero de páginas de memoria
- Operaciones a realizar:
 - Abrir un **objeto de memoria compartida** (**shm_open**)
 - Fijar la longitud del objeto (**ftruncate**)
 - “Mapear” el objeto sobre el espacio de direcciones de proceso (**mmap**)
 - El objeto puede cerrarse, pues es accesible de manera directa (**close**)
 - Cuando un proceso no la utilice, puede eliminarse el “mapeo” (**munmap**)
 - Cuando el objeto ya no sirva, puede destruirse (**shm_unlink**)

```
/* Ejemplo: */
#include <sys/mman.h>
#include <sys/stat.h> /* Modos de acceso, como
                      S_IRWXU */
#include <fcntl.h>    /* Flags, como O_CREAT */
#include <limits.h>

int fd;
int *pmem;
off_t tamaño;

tamaño = sysconf(_SC_PAGESIZE);
fd = shm_open("/memcom",
              O_CREAT | O_RDWR, S_IRWXU);
ftruncate(fd, tamaño);
pmem = (int *)mmap(0, tamaño,
                  PROT_READ | PROT_WRITE,
                  MAP_SHARED, fd, 0);
close(fd);
<....>
```

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

11

Memoria compartida en POSIX 1003.1b

- Prototipo de **mmap**:


```
void *mmap(void *donde, size_t long, int protec, int mapflags, int fd, off_t offset);
```
- Argumentos de **mmap**:
 - **donde**: Dirección de “mapeo” propuesta (puede ser rechazada). Normalmente: 0 (ninguna dirección en particular)
 - **longitud**: Tamaño de la zona “mapeada”; conviene que sea un número entero de páginas. Normalmente: tamaño total del objeto de memoria.
 - **protec**: Combinación de permisos “hardware”, compatibles con los permisos del objeto de memoria. Normalmente: `PROT_READ | PROT_WRITE`.
 - **fd**: Descriptor de archivo asignado al objeto de memoria.
 - **mapflags**: Opciones de “mapeo”: `MAP_SHARED` habilita el acceso desde varios procesos. `MAP_FIXED` obliga a aceptar **donde**, o fallar. Normalmente: `MAP_SHARED`.
 - **offset**: Desplazamiento de la zona mapeada, tomando como referencia el principio del objeto. Normalmente: 0.
 - Valor de retorno:
 - Dirección virtual donde el proceso puede “ver” el objeto de memoria, o -1, si **mmap** falla. Puede ser igual a **donde**, pero no necesariamente.

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

12

Sincronización para hilos (POSIX 1003.1c)

- Sincronización para múltiples hilos (POSIX 1003.1c)
 - Se comparte **automáticamente** todo el espacio de direccionamiento
 - Sincronización para exclusión mútua: **mutex**
 - Sincronización para espera de condiciones: **variable de condición**
 - Opcionalmente el sistema operativo puede permitir usarlos entre hilos de procesos distintos
- **Mutex:**
 - Similar a un **semáforo binario**, que puede estar “abierto” o “cerrado”
 - No se puede:
 - Cerrar un mutex no inicializado
 - Cerrar un mutex que ya se ha cerrado
 - **Abrir un mutex que no se ha cerrado antes**
 - A diferencia de un semáforo, el **hilo que cierra el mutex es el que tiene que abrirlo**

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

13

Mutex

- **Inicialización de un mutex:**
 - Representado por una variable **pthread_mutex_t** (global, reservada con **malloc** en “heap”)
 - Inicialización:
 - Estática (sólo globales): Macro **PTHREAD_MUTEX_INITIALIZER**
 - Dinámica: `int pthread_mutex_init(pthread_mutex_t *mutex, pthread_mutexattr_t *attr);`
- **Cierre de un mutex:**

```
int pthread_mutex_lock(pthread_mutex_t *pmutex);
```

 - Es bloqueante, y la espera no se rompe por señales
 - Si un hilo intenta cerrar un mutex que ya ha sido cerrado por otro hilo, debe esperar a que sea abierto
- **Apertura de un mutex:**

```
pthread_mutex_unlock(pthread_mutex_t *pmutex);
```

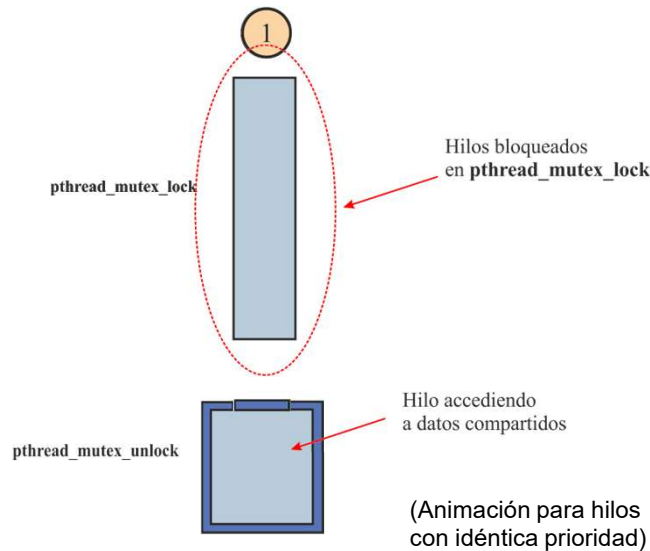
 - Si varios hilos están esperando por el mutex, sólo uno lo consigue cerrar y abandona la espera; el resto sigue esperando
 - Criterio: prioridad y algoritmo de planificación

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

14

Mutex



11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

15

Mutex

```
#include <pthread.h>

/* Inicialización estática */
pthread_mutex_t mut =
    PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t *pmut;
(...)

/* Inicialización dinámica */
pmut = (pthread_mutex_t *)malloc(
    sizeof(pthread_mutex_t));
pthread_mutex_init(pmut, NULL);
(...)

/* Sección crítica de hilo 1 */
pthread_mutex_lock(&mut);
<código de la sección crítica>
pthread_mutex_unlock(&mut);
(...)

/* Sección crítica de hilo 2 */
pthread_mutex_lock(&mut);
<código de la sección crítica>
pthread_mutex_unlock(&mut);
```

- **Otras llamadas:**

- **pthread_destroy** libera los recursos asociados a un mutex.
- **pthread_mutex_trylock**, no es bloqueante: Si el mutex está cerrado, devuelve un error EBUSY y no espera

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

16

Variables de condición

- **Condición:** Expresión con resultado binario que depende de datos compartidos protegidos por un mutex ($A==B$, $A>5$,)
- **Variable de condición:**
 - No es la condición, no es cierta ni falsa, **no tiene estado que indique si se cumple la condición**
 - Es un medio de sincronización. Se utiliza para notificar el cumplimiento de una condición
 - Está **asociada implícitamente al mutex** que protege las variables compartidas de las que depende la condición
- **Creación e inicialización:**
 - Representada por una variable de tipo **pthread_cond_t** (global, reservada con **malloc** en "heap")
 - Inicialización:
 - Estática (sólo globales): Macro PTHREAD_COND_INITIALIZER
 - Dinámica: int **pthread_cond_init**(pthread_cond_t *cond, pthread_condattr_t *attr);

```
#include <pthread.h>
/* Inicialización estática */
pthread_cond_t condicion =
    PTHREAD_COND_INITIALIZER;
pthread_cond_t *pcond;
(...)

/* Inicialización dinámica */
pcond = (pthread_cond_t *)malloc(
    sizeof(pthread_cond_t));
pthread_cond_init(pcond, NULL);
(...)
```

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

17

Variables de condición

- **Invalidación y liberación de recursos:**

```
int pthread_cond_destroy(pthread_cond_t *cond);
```
- **Operaciones básicas:**
 - Esperar activación:
 - El hilo se bloquea a la espera de activación de la variable de condición
 - Una activación anterior a la espera no sirve para desbloquear

```
int pthread_cond_wait(pthread_cond_t *cond,
    pthread_mutex_t *mutex);
int pthread_cond_timedwait(pthread_cond_t *cond,
    pthread_mutex_t *mutex,
    const struct timespec *abstime);
```
 - Activar:
 - Al activar la variable uno o varios de los hilos que esperan activación se desbloquean (si hay alguno)
 - La activación **no se memoriza**; no sirve para desbloquear esperas futuras

```
int pthread_cond_broadcast(pthread_cond_t *cond);
int pthread_cond_signal(pthread_cond_t *cond);
```
 - Para usarlas **es necesario haber cerrado el mutex**

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

18

Espera de una condición

- Código de la espera:

```
#include <pthread.h>
.....
pthread_mutex_t mut = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
.....
pthread_mutex_lock(&mut);
while(<condición (expresión que depende de datos compartidos) es falsa>) /* Mientras cond. falsa */
{
    pthread_cond_wait(&cond, &mut);
}
< procesamiento necesario >
pthread_mutex_unlock(&mut);
```

- Espera con sobretiempo:

- **pthread_cond_timedwait** incorpora un “timeout” expresado en **tiempo absoluto**, no relativo
- Si vence el sobretiempo el hilo ya no espera la condición, y el retorno es ETIMEDOUT
- Sin embargo el hilo **siempre** ha esperado hasta “cerrar” el mutex de nuevo, y debe liberarlo en cualquier caso

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

21

Espera de una condición

- Ejemplo:

- Esperar condición **$i > j$**
- Esperar mientras **no** se cumpla tal condición es esperar **mientras $i \leq j$** (la condición negada de la que se espera)

```
(...)
pthread_mutex_t mut = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int i, j; /* Variables compartidas */
(...)
/*  $i > j$  es una precondición para ejecutar f */
/* Con el mutex cerrado, ni i ni j cambian */
pthread_mutex_lock(&mut);
while( $i \leq j$ ) { /* Iterar mientras la condición es falsa */
    pthread_cond_wait(&cond, &mut); /* Ojo: aquí dentro se abre el mutex, si no... */
}
f(...);
pthread_mutex_unlock(&mut);
(...)
```

- Estamos esperando **la condición $i > j$** utilizando la variable de condición **cond**

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

22

Espera de una condición

- Ejemplo con sobretiempo:

- Esperar condición $i == 5 \parallel j == 3$ con sobretiempo de 1 segundo
- Hay que esperar mientras $!(i==5 \parallel j==3)$, o sea $j!=5 \&\& j!=3$

```
(...)
pthread_mutex_t mut = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int i, j; /* Variables compartidas */
struct timespec ahora; int r;
(...)
/* i==5 || j==3 es una precondition para ejecutar f */
pthread_mutex_lock(&mut);
clock_gettime(CLOCK_REALTIME, &ahora);
ahora.tv_sec += 1; r = 0;
while(i != 5 && j != 3 && r == 0) { /* Iterar mientras cond. falsa */
    r = pthread_cond_timedwait(&cond, &mut, &ahora);
}
if(r==0) f(...);
else tratar_error(...);
pthread_mutex_unlock(&mut); /* En cualquier caso, abrir mutex */
(...)
```

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

23

Activación de una variable de condición

- Consideraciones generales:

- Si un hilo está bloqueado en **pthread_cond_wait**, otro debe activar la variable para salir de tal situación
- El hilo “activador” puede ser el **causante** de que se cumpla de nuevo la condición, o simplemente ha **detectado su cumplimiento**
- Normalmente el “activador” ha “cerrado” el mutex, y **debe liberarlo** más tarde
- Cuando un hilo es desbloqueado por la activación, **aún tiene que competir por el mutex** antes de salir de la espera – se mantiene la exclusión mutua

- Procedimiento habitual:

```
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
pthread_mutex_t mut = PTHREAD_MUTEX_INITIALIZER;
(...)
pthread_mutex_lock(&mut);
<operaciones que originan el cumplimiento de la condición>
pthread_cond_broadcast(&cond); /* o pthread_cond_signal */
(...)
pthread_mutex_unlock(&mut);
```

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

24

Activación de una variable de condición

- Opciones para activar:
 - pthread_cond_broadcast:** Todos los hilos son desbloqueados.
 - Ningún hilo queda bloqueado cuando se cumple la condición, pero muchos pueden “despertarse” inútilmente
 - pthread_cond_signal:** Al menos uno de los hilos es desbloqueado
 - No se desbloquean hilos inútilmente, pero es posible que algunos se queden bloqueados aunque la condición se cumpla
 - Soluciones para evitarlo:
 - Si sabemos el número adecuado, realizar varias activaciones
 - Otra opción: hacer que cualquier hilo active la variable cuando compruebe que la condición se cumple, aunque no sea el causante:

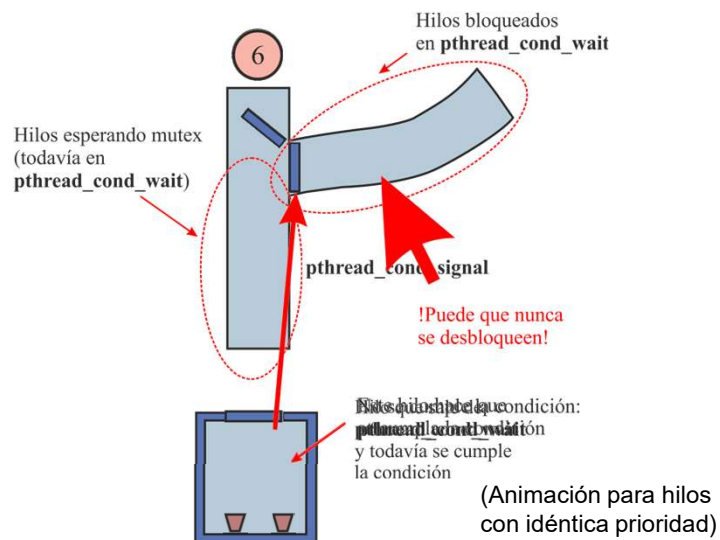

```
pthread_mutex_t mut = PTHREAD_MUTEX_INITIALIZER;
(...)
/* En cualquier hilo que acceda a los datos compartidos */
pthread_mutex_lock(&mut);
<operaciones necesarias>
<comprobar condiciones; para las que se cumplan, hacer
pthread_cond_signal>
pthread_mutex_unlock(&mut);
```
 - Orden de desbloqueo: Prioridad y algoritmo de planificación

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

25

Variables de condición



Activación con pthread_cond_signal y posibles problemas

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

26

Regiones críticas condicionales

- **Regiones críticas condicionales**
 - Los datos compartidos se agrupan en recursos, con nombre conocido
 - El lenguaje permite definir las **regiones críticas**, relacionándolas con un determinado recurso
 - El sistema asegurará la ejecución de las regiones críticas relacionadas con el mismo recurso en **exclusión mutua**
 - La sincronización con condiciones se resuelve con **"guardas"**

Recurso A:

<definición de variables compartidas>
int i;

Fin A.

Tarea P1:

<...>
Region A, cuando i!=0
<región crítica>
Fin Region.
<...>

Fin P1

- **Solución poco estructurada**
- **Reevaluación de condiciones:** elevado coste en conmutaciones de procesos
 - La tarea que cambia i... no "sabe" que tiene que avisar a P1

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

27

Monitores

- **Principios de funcionamiento:**
 - Datos compartidos, inaccesibles de manera directa
 - Funciones que permiten realizar operaciones con los datos compartidos con acceso exclusivo
 - Sincronización con condiciones: operaciones **wait** y **signal**
- **Conclusiones:**
 - Encapsulamiento de datos y procedimientos
 - Mezcla de características de alto y bajo nivel (**wait** y **signal**) poco deseable
 - Servicios de **mutex** y **variables de condición** (POSIX 1003.1c) pensados para construir monitores

monitor A:

condiciones:

noescero;
nomax;

datos:

<...>
int MAX = 50;
int i;

fin datos;

entrada dec:

if(i==0) wait(noescero);
i=i-1;
signal(nomax);
<...>

fin dec;

entrada inc:

if(i==MAX) wait(nomax);
<...>
i = i + 1;
signal(noescero);

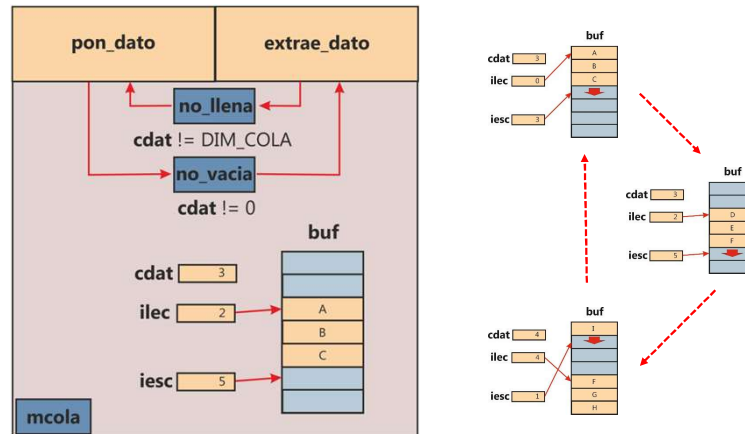
fin inc;

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

28

Monitor con POSIX: Cola circular



Monitor para una cola circular. **mcola** es un mutex;
no_llena y **no_vacia** son variables de condicion

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

29

Objetos protegidos (ADA)

- **Principios de funcionamiento:**
 - Los datos compartidos solo son accesibles a través de procedimientos.
 - El acceso a los procedimientos del objeto protegido se realiza con garantías de exclusión mutua.
 - La sincronización con condiciones se realiza mediante barreras de entrada (“barriers”)
- **Acceso al objeto protegido:**
 - **function:** Acceso concurrente de varias tareas, con “**read lock**”: no se pueden modificar los datos compartidos; no se permite el acceso a **procedure** o **entry** hasta que quede libre el recurso.
 - **procedure:** Pueden modificarse los datos compartidos, y se garantiza el acceso exclusivo; el recurso está bajo “**read/write lock**”.
 - **entry:** Llevan asociada una condición (“**barrier**”) que debe verificarse, además de estar libre el recurso.

```
protected type ejemplo is
  function lee return Integer;
  procedure reset;
  entry dec(decre: in Integer);
  entry inc(incre: in Integer);
private
  midato: Integer := 0;
  maximo: Integer := 50;
end;

protected body ejemplo is
  function lee return Integer is
  begin
    return midato;
  end lee;
  procedure reset is
  begin
    midato := 0;
  end reset;
  entry dec(decre: in Integer)
  when midato > 0 is
  begin
    midato = midato - decre;
  end dec;
  entry inc(incre: in Integer)
  when midato <= maximo is
  begin
    midato = midato + incre;
  end inc;
end ejemplo;
```

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

30

Objetos protegidos (ADA)

- Evaluación de las condiciones de barrera:
 - Una tarea llama a una entry, y existe la posibilidad de que la condición se haya modificado desde su última evaluación.
 - Una tarea sale del objeto protegido, existen tareas encoladas a causa de una barrera que no se verifica y existe la posibilidad de que la tarea que abandona el objeto haya provocado el cambio de su condición asociada.

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

31

Variables compartidas: Puntos a recordar

- Sincronización para exclusión mutua:
 - Como regla general, **no** se deben realizar operaciones con variables compartidas sin exclusión mutua:
 - Cambiar el valor
 - Comprobar el valor
 - Realizar cálculos
 -
 - Excepción: Innecesario para variables que no cambian
 - Sección crítica: En general debe ser lo más breve posible (se puede estar impidiendo la concurrencia)
- Mutex:
 - Permiten garantizar la exclusión mutua
 - Los mutex no pueden usarse para esperar condiciones
 - ¿Cuántos hay que usar?
 - Uno por grupo de datos que hay que considerar conjuntamente
 - Si usamos menos, el resultado no es óptimo (se impide la concurrencia innecesariamente), pero funciona...

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

32

Variables compartidas: Puntos a recordar

- **Variables de condición:**
 - **Nunca** deben activarse o esperarse variables de condición sin tener el mutex
 - La variable de condición **no** es la condición, ni tiene un valor asociado
 - Las activaciones pasadas no desbloquean la espera, sólo las futuras pueden hacerlo (no se memorizan...)
 - En general debe usarse el procedimiento recomendado (u otro equivalente):
 - Nunca esperar si la condición ya se cumple
 - Comprobar tras salir de la espera si la condición aun se cumple
 - La condición de mantenimiento del **while** es la **negada** de la que esperamos
 - Sobretiempo para la espera:
 - No podemos interrumpirla con una señal
 - Debemos usar **pthread_cond_timedwait** o activar variable de condición desde otro hilo

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

33

Variables compartidas: Puntos a recordar

- **pthread_cond_wait:**
 - El mutex es el que protege los datos de los que depende la condición
 - **pthread_cond_wait**(&mut, &cond) se parece a hacer


```
pthread_mutex_unlock(&mut);
esperar_activación(&cond); /* Llamada imaginaria... */
pthread_mutex_lock(&mut);
```
- **pthread_cond_timedwait:**
 - Ojo: El sobretiempo se especifica con un tiempo **absoluto**, no con un intervalo
 - Incluso si hay sobretiempo, el hilo tiene el mutex al volver

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

34

Variables compartidas: Puntos a recordar

- **pthread_cond_signal**
 - Es apropiada si sólo hay un hilo que espera la condición
 - También si la condición sólo “le vale” a uno de los hilos que esperan
 - Por ejemplo, el hilo que sale de la espera hace que la condición no se cumpla (la condición “se consume”...)
 - Un caso concreto: Un hilo envía un conjunto de datos que sólo otro (posiblemente entre varios) debe recibir
 - Si sabemos que la condición “vale” para N hilos, también podemos invocar **pthread_cond_signal** N veces
 - Un hilo que no se desbloquea cuando debe puede ser un problema grave
- **pthread_cond_broadcast**
 - Es apropiada cuando sabemos que la condición “vale” para todos los hilos que puedan estar esperando
 - Por ejemplo: Los hilos que salen de la espera no afectan al cumplimiento de la condición (la condición “no se consume”)
 - Un caso concreto: Respuesta ante una alarma, que exige la activación de varios hilos a la vez
 - O también si no sabemos realmente a cuantos hilos puede “valer” la condición
 - Un hilo que se desbloquea sin necesidad sólo disminuye la eficiencia

11/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

35